

Machine Learning in Nonlinear Dynamical Systems

Sayan Roy*

*Department of Physics,
Indian Institute of Science Education and Research Bhopal,
Bhopal Bypass Road, Bhauri, Bhopal, Madhya Pradesh, 462066, India*

Debanjan Rana†

*Department of Chemistry,
Indian Institute of Science Education and Research Bhopal,
Bhopal Bypass Road, Bhauri, Bhopal, Madhya Pradesh, 462066, India*

Abstract

In this article, we discuss some of the recent developments in applying machine learning (ML) techniques to nonlinear dynamical systems. In particular, we demonstrate how to build a suitable ML framework for addressing two specific objectives of relevance: prediction of future evolution of a system and unveiling from given time-series data the analytical form of the underlying dynamics. This article is written in a pedagogical style appropriate for a course in nonlinear dynamics or machine learning.

1 Introduction

Study of dynamics has fascinated mankind for many centuries. One of the early things that intrigued the human mind was the motion of objects, both animate and inanimate [1]. Interest and curiosity in understanding the motion of planetary objects and various natural phenomena such as wind, rain, etc. led to the development of the field of nonlinear dynamics as a major branch of study in physics and mathematics, with potential applications in different branches of science and engineering [2].

Nonlinear dynamics has played a crucial role in understanding complex systems. One is routinely confronted with new phenomena warranting suitable dynamical modelling. In many cases, however, even if such a modelling may be achieved, exact closed-form solutions of the dynamics remain elusive with existing mathematical techniques.

With vast amount of data being generated and enhanced data management systems in the world of today, we have tons of data to analyze and exploit to our advantage. In this regard, developing a method that may predict future events by learning from past data would evidently be considered a major advancement. For example, making reliable predictions for the closing price of stock each day, the issue of occurrence of cardiac arrhythmia from existing ECG data, the long-term dynamical state for a chaotic system would be highly desirable. Machine learning algorithms fundamentally work on a similar strategy of learning from given data, and have proven to be very efficient in finding patterns from higher-dimensional data

*sayanr16@iiserb.ac.in

†debanjan16@iiserb.ac.in

such as those involving images, speech, etc. [3]. In 1998, Tom Mitchell in his textbook [4] gave a very logical definition of machine learning, defining it as a Well-posed Learning Problem. He writes, “A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T , as measured by P , improves with experience E ”. By following this definition, we propose in this contribution an ML framework that may predict future data by learning from existing data. For our model, E is the given time-series data that we feed into the algorithm, T is the task of prediction, and P measures the performance of whether it can predict correctly. We will present in this work an explicit application of this technique in understanding representative dynamical systems for which analytical closed-form solution is not available.

Another direction that fascinates us about machine learning is the associated data-driven discovery of the governing dynamical equations. Traditionally, nonlinear dynamics has progressed by invoking fundamental principles and intuitions in developing theoretical explanations of observations. In the present big data era, a central challenge is to reconstruct the underlying dynamical system from an analysis of the existing data. In this backdrop, we discuss here a novel technique called Sparse Identification of Nonlinear Dynamical Systems (SINDy) [5], developed by *Brunton et al.* based on an ML concept called sparse regression, which is used to unveil governing dynamical laws from time-series data. Underlying the technique is the reasonable assumption that most physical systems have simple dynamics containing only a few of the many nonlinear functions possible, so that the governing equation becomes sparse in the high dimensional space of nonlinear functions. We will demonstrate here an application of the algorithm in the context of two paradigmatic nonlinear oscillators, showing in particular how it captures in a very efficient manner the rich nonlinear behavior of the two systems.

This article is laid out as follows: In Section 2, we describe the machine learning framework that predicts future data by learning from existing data, which we then apply to two different time-series data. In Section 3, we introduce the SINDy algorithm followed by its application to two paradigmatic nonlinear oscillator systems, namely, the Duffing-Van der Pol oscillator and the Rössler attractor. The article ends with key concluding remarks in the last section, followed by an appendix that contains some technical details.

2 Prediction with Neural Networks

In order to start our discussion, let us consider a time-series data of the form $x(t_1), x(t_2), \dots, x(t_n)$, where $x(t_i)$ represents the value of the dynamical variable x at i -th time instant t_i and n is the total number of data points in the time-series data. We first split the dataset into a training set (70 % of total data) and a test set (30 % of total data). The ML model is trained in the training set, and then we evaluate the goodness of the model by comparing its predictions in the test set with the original test set data. The time-series dataset is restructured in the form shown in Table 1.

This type of restructuring makes it evident that the output data at a given time is determined in terms of input data from all previous times over a given time interval (the so-called sliding time window). This type of ML setting is known as **supervised learning**. Mathematically, we define supervised learning as follows. Let $S \equiv (x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_N, y_N)$ be the dataset, where N is the number of data points in this restructured format, x_i ’s define the input dataset X and y_i ’s constitute the output dataset Y . When Y is continuous, one talks of a regression problem, while for discrete Y , one has a classification problem. For example, referring to Table 1, we have $x_1 \equiv \{x(t_1), x(t_2), \dots, x(t_{20})\}$ and $y_1 \equiv x(t_{21})$. Now, there exists a function $F : X \rightarrow Y$ which satisfies all the data points in S but its analytical form is unknown. Machine learning aims to identify this mapping. To this end, we choose an ML framework, and by using a suitable learning algorithm, we check with the help of an accuracy metric whether

Input Data	Output Data
$x(t_1), x(t_2), x(t_3), x(t_4), \dots, x(t_{20})$	$x(t_{21})$
$x(t_2), x(t_3), x(t_4), x(t_5), \dots, x(t_{21})$	$x(t_{22})$
$x(t_3), x(t_4), x(t_5), x(t_6), \dots, x(t_{22})$	$x(t_{23})$
$x(t_4), x(t_5), x(t_6), x(t_7), \dots, x(t_{23})$	$x(t_{24})$
$x(t_5), x(t_6), x(t_7), x(t_8), \dots, x(t_{24})$	$x(t_{25})$

Table 1: Restructuring the dataset for neural networks. We choose 20 as the length of the sliding window. Here, the number of data points N used is 6880. A glimpse of the dataset containing 5 elements is shown here.

the framework can approximate well the true function. The ML framework and the learning algorithm together form the *ML model*. In our setting, the ML framework is a neural network, while the learning algorithm is that of gradient descent.

Neural Network Architecture

The concept of neural networks (NNs) is inspired from biological neurons. A visual representation of an NN is shown in Fig. 1. The first layer is the input layer, and each orange box contains an input data point. The last layer is called the output layer, while all the intermediate layers are called the hidden layers. The hidden layers are constituted by nodes, shown by blue circles in Fig. 1, which are also termed as artificial neurons/ perceptrons. The nodes are the building blocks of the neural network architecture. Each node in the first hidden layer receives a set of inputs $\{x(t_1), x(t_2), \dots, x(t_d)\}$ (with d being the length of the sliding time window) from the previous layer, and these are multiplied with their corresponding weights $\{w_1, w_2, \dots, w_d\}$ and summed up. A bias term w_0 is added to the sum, which is then passed through an activation function to get the output $\hat{y}$ of that node. The bias term acts as an additional parameter that helps to adjust the output and adds flexibility to the learning process. The purpose of the activation function is to introduce non-linearity into the model, thereby allowing modeling of the output as a nonlinear function of the input data. We thus have

$$\hat{y} = g \left(w_0 + \sum_{i=1}^d w_i x_i \right), \quad (1)$$

where g is the activation function. The output from each node is then passed on as the input to the next layers in the forward direction until one reaches output layer. In this way, by forming a network of perceptrons, a neural network gets constructed. The weights and biases for all the nodes taken together are referred to as the *parameters* (W) of the NN model. Parameters other than weights and biases are termed as *hyperparameters*, e.g. the number of hidden layers, number of neurons (nodes) in a layer, learning rate, and many more [3]. After building this NN model, we construct a loss function $L(W)$, as

$$L(W) \equiv \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i, W))^2, \quad (2)$$

where y_i is the actual output data, $f(x_i, W)$ is the predicted output with x_i representing the input data, and N is the number of input data points. The physical significance of loss function is that it is a measure of the prediction error of the ML framework. Now, we learn the parameters W^* such that it satisfies

$$W^* = \min_W L(W). \quad (3)$$

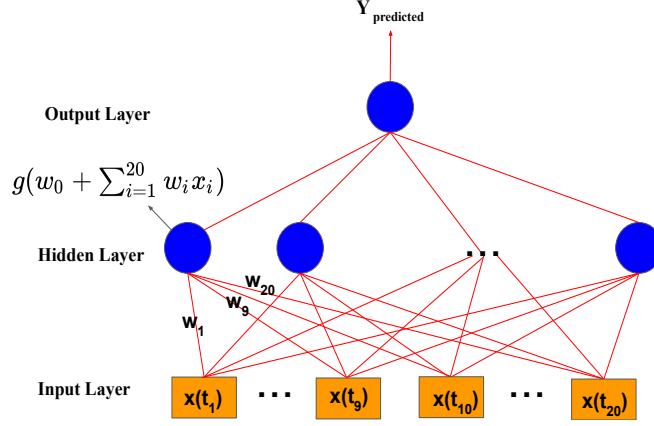


Figure 1: Neural Network Architecture consisting of three types of layers: an input layer, several hidden layers, and an output layer. The boxes and the circles represent the nodes of the network, while a red line joining two nodes represents the weight parameter for the corresponding connection and g the activation function. For more details, see text.

This optimization may be done by using a gradient descent algorithm discussed below.

Algorithm 1: Gradient Descent Algorithm

Initialize weights randomly, $W \sim \mathcal{N}(0, \sigma^2)$
while *Until convergence i.e. Loss Function $\geq \epsilon$* **do**
 compute gradient, $\frac{\partial L(W)}{\partial W}$
 update weights, $W \rightarrow W + \eta \frac{\partial L(W)}{\partial W}$
end
return W

In each iteration, the algorithm computes the gradient of the loss function with respect to all the parameters $\partial L(W)/\partial W$ by a method called *backpropagation* [3]. The parameters are then updated with their respective gradients. This is how learning proceeds in an NN model. Once the learning is achieved, if we feed the test data into the NN model, it should give the expected output with good accuracy. The parameter η , called the *learning rate*, is responsible for the convergence of the algorithm.

2.1 Results and Discussions

Root Mean Square Error (RMSE) was selected as the metric to evaluate the performance of the neural network model. It is defined as

$$\text{RMSE} \equiv \sqrt{\frac{\sum_{i=1}^N (y_i^{\text{pred}} - y_i^{\text{actual}})^2}{N}}. \quad (4)$$

We trained multiple NN models simultaneously by varying the number of hidden layers, number of nodes in a layer, learning rate and other *hyperparameters*. The model which has the minimum RMSE on the test set is the best model. We have implemented our NN model¹ in two different time-series data corresponding to two different systems. The first dataset (Dataset-I) corresponds to the Lorenz system, whose governing equations [6] are

$$\dot{x} = \sigma(y - x), \quad \dot{y} = x(\rho - z) - y, \quad \dot{z} = xy - \beta z, \quad (5)$$

¹We have used a python package called TensorFlow for performing the numerical computations.

Dataset	Best NN Model	Learning Algorithm	RMSE
Dataset-I	20-100-10-1	GD	0.38 %
Dataset-II	20-128-64-64-1	GD	3.88 %

Table 2: Description of the NN models for the two data sets. The NN architecture is written in the order of the number of nodes in each layer. GD means gradient descent. We have used a modified version of GD called stochastic gradient descent with momentum 0.9 [3] to perform our numerical computations.

with ρ , σ and β being the parameters of the system. For values $\sigma = 10$, $\rho = 28$, $\beta = 8/3$, the system exhibits chaotic behavior [6]. The data were obtained by numerically integrating the Lorenz equations by fourth order Runge Kutta method with parameters in the chaotic regime. We consider the x data to be our input data. After training the NN model, we plot in Fig. 2 the predicted evolution of the state x versus time. For comparison, we also plot the actual dynamics in this figure. Clearly, we observe that the yellow dashed line (predicted dynamics) approximates perfectly the blue solid line (actual dynamics). From our analysis, the model was found to be giving 0.38 % RMSE error in the test set. The neural network model knows nothing a priori about the governing equations of the data, and yet, it is able to learn very well from the training data and predict the data in test set i.e., future motion. Recently, advanced algorithms similar to NN like Reservoir computing have shown great success in predicting the long term evolution of chaotic systems [7].

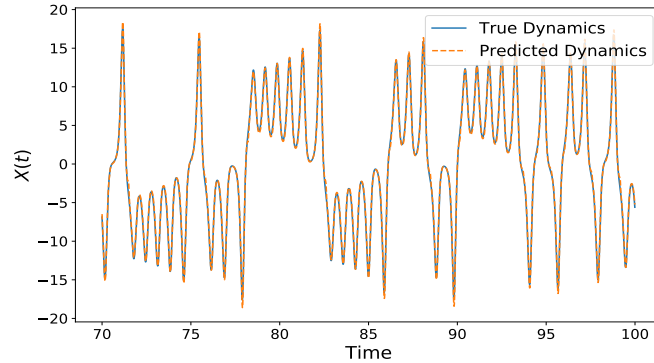


Figure 2: For the x -coordinate of the Lorenz system (5), the figure shows the predictions in the test set from the best NN model (dashed orange line) compared with those from numerical integration of the actual dynamics (continuous blue line). The learning rate is $\eta = 10^{-6}$, while Table 2 shows the NN architecture.

The second dataset (Dataset-II) is a noisy time-series data depicting a more practical scenario. We generate these data by adding random numbers (noise) to a periodic time-series data with the average increasing in time. The results are shown in Fig. 3 with the best model having a 3.88 % error. We observe from the figure that the NN model has successfully captured the trend, seasonality, and noisy areas of the actual time-series data. Although we do not have any dynamical model for this time-series data, the NN model is able to predict the dynamics sufficiently well. This feature of learning from a finite amount of training data and predicting quite accurately the data in the test set makes this ML technique quite a powerful and useful tool. All of the aforementioned results and the architecture of the best NN models are summarized in Table 2.

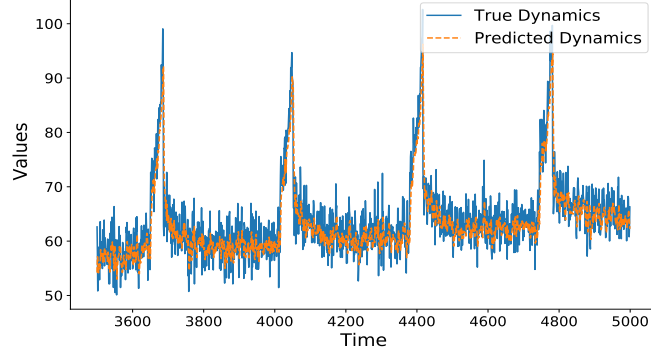


Figure 3: The best model of standard neural network in the Test Set for a generated stochastic time-series data. The solid blue line is the actual dynamics and the dashed orange line is the predicted dynamics from the NN model. The learning rate is $\eta = 10^{-6}$. NN architecture is shown in Table 2.

3 Sparse Identification of Nonlinear Dynamical Systems

The idea of SINDy [5] (Sparse Identification of Nonlinear Dynamical Systems) is to obtain the governing equations of a nonlinear dynamical system with d degrees of freedom from the time-series data of the dynamical state $\mathbf{x}$ defined as,

$$\mathbf{x} \equiv [x_1 \ x_2 \ \dots \ x_d]. \quad (6)$$

Specifically, the objective here is to obtain a dynamical equation of the form

$$\frac{d\mathbf{x}(t)}{dt} = \mathbf{f}(\mathbf{x}(t)). \quad (7)$$

Here, the function $\mathbf{f} \equiv [f_1 \ f_2 \ \dots \ f_d]$ determines the dynamical evolution of $\mathbf{x}(t)$. Now, it is a fact that for most known dynamical systems, the governing equations contain only a few functions among all possible functions. To understand this statement, consider for example the Duffing oscillator, which has only linear and cubic terms in its equations of motion:

$$\dot{x} = y, \quad \dot{y} = -x - \beta x^3. \quad (8)$$

In matrix form, the above dynamics may be rewritten as

$$\underbrace{\begin{bmatrix} \dot{x} & \dot{y} \end{bmatrix}}_{\dot{\mathbf{x}}} = \underbrace{\begin{bmatrix} 1 & x & y & x^2 & xy & y^2 & x^3 & x^2y & xy^2 & y^3 \end{bmatrix}}_{\Theta(\mathbf{x})} \underbrace{\begin{bmatrix} 0 & 0 \\ 0 & -1 \\ 1 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & -\beta \\ 0 & 0 \\ 0 & 0 \end{bmatrix}}_{\underbrace{\begin{bmatrix} \underbrace{0}_{w_1} & \underbrace{0}_{w_2} \end{bmatrix}}_W}, \quad (9)$$

where $\Theta(\mathbf{x})$ is the library of polynomial functions in (x, y) up to third order. Clearly, the weight matrix $W \equiv [w_1 \ w_2]$ for the Duffing oscillator is sparse (most of its elements are 0). Thus we

assume that the equations of motion for any system involve only a few functions among all possible elementary functions represented by the library:

$$\Theta(\mathbf{x}) = [\theta_1(\mathbf{x}) \quad \theta_2(\mathbf{x}) \quad \dots \quad \theta_p(\mathbf{x})] \quad (10)$$

where p is the total number of candidate functions and θ_a with $a = 1, \dots, p$ denotes nonlinear functions in $\mathbf{x}$. Note that $\theta_a(\mathbf{x}) \equiv \theta_a(x_1, x_2, x_3, \dots, x_d)$. One may choose the set of functions $\theta_a(\mathbf{x})$ according to the system of interest.

Each column equation in (7) represents the time evolution of a particular component of $\mathbf{x}$ (such as $\dot{x}$ and $\dot{y}$ for Duffing oscillator (8)). This may be written as

$$\frac{dx_k(t)}{dt} = \Theta(\mathbf{x})w_k \quad \text{for } k = 1, \dots, d \quad (11)$$

where w_k corresponds to the weight vector consisting of the weights for all the p functions in $\Theta(\mathbf{x})$. We compute $\frac{dx_k(t)}{dt}$ and $\Theta(\mathbf{x})$ based on the given time-series data. The time-derivative of the time-series data may be obtained either from measurements or by using suitable numerical differentiation techniques. To determine w_k , we have to construct a loss function in such a way that the weight vector w_k is sparse.

This is a supervised learning framework, where $\dot{x}_k$ is the actual output data y_i and $\Theta(\mathbf{x})w_k$ is the predicted value of the output from the SINDy model, similar to $f(x_i, W)$ in the NN model. In both cases, we learn the parameter set through regression. The loss function which results in a sparse parameter set has the form

$$L_1(W) = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i, W))^2 + \lambda \sum_{j=1}^M |w_j|, \quad (12)$$

where N is the total number of data points, M is the total number of parameters and λ is a regularization constant. We have discussed intuitively in the appendix why such a loss function is appropriate for learning sparse parameter set.

Once the numerical values of the elements of the weight vector are determined, the governing equation for each column may be constructed as $\dot{x}_k = \Theta(\mathbf{x})w_k$ for $k = 1, \dots, d$ as in Eq. (9).

3.1 Results and Discussions

In this section, we discuss results obtained on application of the SINDy method to two representative nonlinear dynamical systems. The time-series data required for our purpose are obtained by employing the usual fourth-order Runge-Kutta method to integrate numerically the defining equations of motion.

The Duffing-Van der Pol Oscillator

The Duffing-van der Pol oscillator [8] is a particular nonlinear oscillator that has been studied extensively over the years due to its rich bifurcation behavior and limit-cycle dynamics. The governing dynamics of the oscillator is given by

$$\ddot{x} - \mu(1 - x^2)\dot{x} + x + \beta x^3 = 0, \quad (13)$$

where μ and β are real constants representing dynamical parameters. The above dynamics may be written down as two coupled first-order differential equations:

$$\dot{x} = y, \quad \dot{y} = \mu(1 - x^2)y - x - \beta x^3. \quad (14)$$

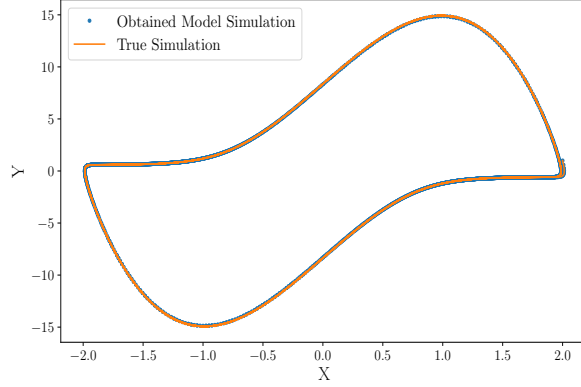


Figure 4: Comparison of the dynamical behavior of the Duffing-Van der Pol oscillator given by Eq. (14) (denoted by the orange continuous line), and the SINDy model given by Eq. (15) (denoted by blue dots).

In our analysis, we have used $\mu = 10$, $\beta = 2$. From the SINDy algorithm, we obtain the dynamics: ²

$$\dot{x} = 0.998y, \quad \dot{y} = -1.002x + 9.914y - 1.995x^3 - 9.914x^2y. \quad (15)$$

We thus see that with the SINDy algorithm, the dynamical parameters have been determined within 0.47 % of their actual values. Moreover, from Fig. 4, we may observe that the predicted dynamics captures quite successfully the actual behavior.

The Rössler attractor

Next, we check how the SINDy method performs in the context of a nonlinear dynamical system displaying a so-called chaotic attractor. We choose the Rössler attractor [9] as our system of study. The governing equations are

$$\dot{x} = -y - z, \quad \dot{y} = x + ay, \quad \dot{z} = b + z(x - c), \quad (16)$$

where the real constants a , b , c are the dynamical parameters of the system. In our analysis, we have chosen the parameter values $a = 0.2$, $b = 0.2$, $c = 5.7$ for which the Rössler system is known to show chaotic behavior [9]. The SINDy algorithm predicts the dynamics as

$$\begin{aligned} \dot{x} &= -1.000y - 1.000z, \\ \dot{y} &= 1.000x + 0.200y, \\ \dot{z} &= 0.200 - 5.697z + 1.000xz. \end{aligned} \quad (17)$$

Chaotic systems are very sensitive to initial conditions: minute perturbations lead to very different dynamics [2]. The dynamical parameter values determined by SINDy differ from their actual values by 0.05 %. This is the reason why at long times, the model trajectory deviates slightly from the true trajectory, as shown in Fig. 5. Nevertheless, from Fig. 6, it is evident that the model captures qualitatively the dynamics of the Rössler attractor. Namely, had we not labeled the two plots in Fig. 6, one would not have been able to make out as to which one corresponds to the original model and which one to the model predicted by SINDy.

The above examples serve to establish the fact that the SINDy algorithm can capture faithfully the actual behavior of two different nonlinear systems, and thus may prove helpful in

²We use a python package called pySINDy for performing the numerical computations.

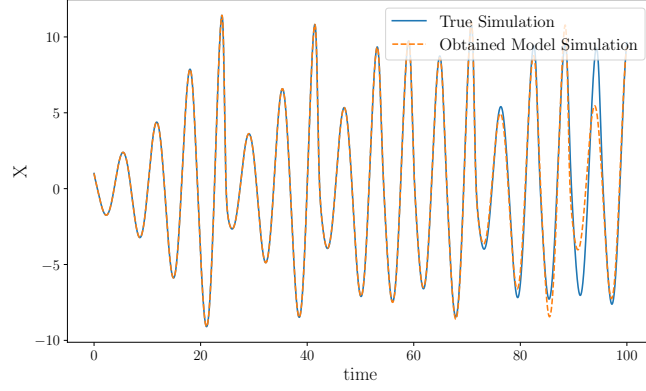


Figure 5: Comparison of X -coordinate time-series data for Rössler attractor (blue solid) and the SINDy model (yellow dashed).

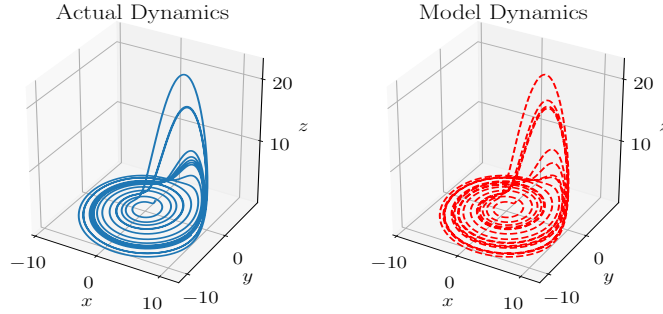


Figure 6: Comparison of the actual trajectory of the Rössler attractor (left) and the trajectory of the model obtained by SINDy (right).

modelling dynamics from given time-series data. The power of this method may be appreciated from an ML study pursued for a model system, namely, the fluid flow example of vortex shedding behind an obstacle. This problem took almost three decades for scientists to unveil its underlying dynamical equations, while based on time-series data, the dynamics could be predicted by SINDy within a few minutes [5]!

Conclusions

In conclusion, we have demonstrated using paradigmatic nonlinear systems the use of two different machine learning framework in predicting and extracting governing dynamical equations from time-series data. We have shown that neural network models can prove powerful in predicting the future evolution of dynamical systems from given data, in cases for which the exact functional form of the governing equations is not known. The SINDy method that we have discussed not only identifies the nonlinearities of the dynamics, but also determines the dynamical parameters with high precision, and the obtained models capture efficiently the nonlinear behavior of the original dynamical system.

Acknowledgement

Sayan Roy acknowledges DST-INSPIRE, Government of India for providing him with a scholarship. Debanjan Rana acknowledges KVPY, Government of India for providing him with a scholarship. The authors are grateful to Shamik Gupta for critically reading the manuscript and for insightful comments on its content. SR also acknowledges Debraj Das for help with figures.

Appendix: Regularization

In this appendix, we discuss in the context of SINDy algorithm introduced in Section 3 how one may construct a loss function that results in a sparse parameter set (most of the parameter values being 0). For simplicity, we will take the number of parameters to be two, though the discussion here is easily generalizable to the case of a larger number of parameters. As already mentioned in Section 2, during the learning process, the loss function has to be minimized with respect to the parameters W . While Fig. 7(a) shows the loss function in the two-dimensional parameter space $\{w_1, w_2\}$, its projection onto the $w_1 - w_2$ plane is represented by a closed contour several of which are shown in Fig. 7(b).

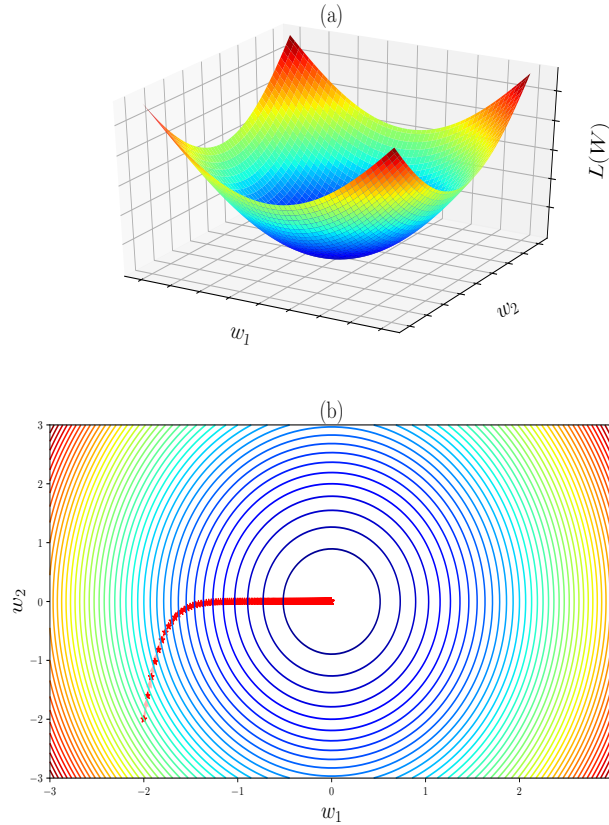


Figure 7: Referring to the discussion in the appendix, (a) is the schematic representation of the loss function $L(W)$, with w_1 and w_2 being the two parameters. At the center of the depicted surface lies the minimum of the loss function. (b) represents the projection of $L(W)$ onto the $w_1 - w_2$ plane. Every closed contour corresponds to a given value of $L(W)$. The red dots represent schematically how the parameters learn about the minimum via gradient descent.

Each contour corresponds to a different value of the loss function. The outer contours have

a larger value of the loss function with respect to the inner contours. The goal of optimization is to learn the parameter values that result in the loss function having its minimum value, with the latter corresponding to the center of the contours. Note that the center does not necessarily correspond to several parameters having zero value. Such an optimization might also “overfit” the model in the training data. Overfitting implies that the model would not predict efficiently for new examples, thereby severely limiting its applicability.

In order to circumvent the problem of overfitting, we apply regularization. The idea is to learn the parameters for which the error is not precisely minimum (Point (a) in Fig. 8(b)), but has a value corresponding to a nearby contour (Point (b) in Fig. 8(b)). The most common is the L2 regularization, defined as

$$L_2(W) \equiv \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i, W))^2 + \lambda \sum_{j=1}^M (w_j)^2, \quad (18)$$

where M is the total number of parameters and λ is the regularization constant. For $M = 2$, Eq. (18) is the mathematical statement of minimizing the loss function $L(w)$ (Eq. (3)) subject to the constraint $w_1^2 + w_2^2 \leq t$, where t is a positive constant [10]. Indeed, it is easy to see that values of w that satisfy both equation Eq. (3) and the constraint would satisfy Eq. (18) for any λ . Geometrically, the constraint represents the area inside the circle $w_1^2 + w_2^2 = t$, see Fig. 8. The parameters that satisfy both the conditions would correspond to the point at which both the contour and the circle have a common tangent (point b in Fig. 8(b)). An outer contour than the one corresponding to this point will satisfy the constraint condition but will have a larger value of the loss function, while an inner contour will not satisfy the constraint condition. For the parameter set to be sparse, the tangent line has to be parallel to the x -axis (shown in Fig. 8(b)) or the y -axis. Any other tangent line (shown in Fig. 8(a)) will result in a non-sparse parameter set. It is then evident the parameters learned this way will have a rather low likelihood of being sparse.

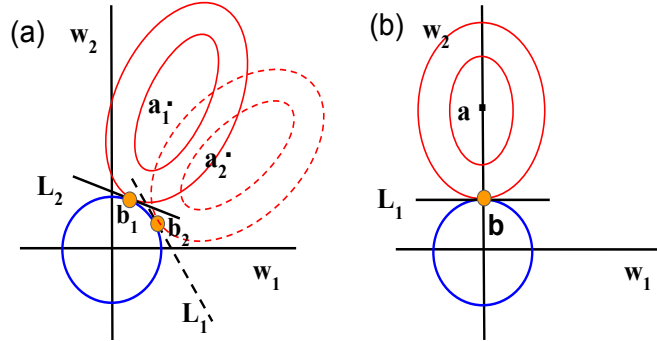


Figure 8: Referring to the discussion in the appendix, (a) represents two cases for which the tangent line is not parallel to either the x or the y axis. The points b_1 and b_2 yield the minimum value of loss function $L_2(W)$. They are not sparse. The case of learned parameters to be sparse is shown in (b). The figures are only schematic.

Another type of regularization is L1 regularization, where the loss function is defined by Eq. (12). Here, for $M = 2$, Eq. (12) is derived from the constraint condition $|w_1| + |w_2| \leq \text{constant}$ [10]. This condition corresponds to the area covered by the diamond shown in Fig. 9. Here, the possible case for not having a sparse parameter set corresponds to the aforementioned tangent line being parallel to any of the edges of the diamond, as shown in Fig. 9(b). Any other tangent

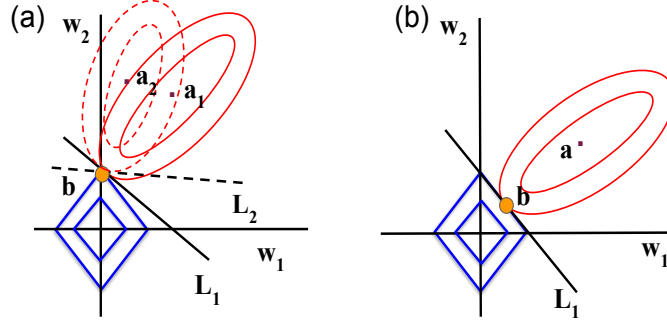


Figure 9: Referring to the discussion in the appendix, (a) represents the case in which the tangent is not parallel to the edges of the diamond. In such cases, the corners of the diamond represent the minimum value of $L_1(W)$, leading to sparsity in learned parameters. One of the cases of non-sparsity is shown in (b). The figures are only schematic.

line such as the one shown in Fig. 9(a) will intersect the diamond at the x -axis or the y -axis corresponding to a minimum value of the loss function. In this case, the learned parameters will have a very high likelihood of being sparse.

On the basis of the above discussions, we intuitively understand that a curve with corners such as a diamond has a higher probability of generating a sparse parameter set than a smooth curve such as a circle. For L1 regularization, one would have more corners with the increase in the number of parameters and consequently a sparser parameter set. A rigorous mathematical demonstration of convergence of L1 regularization to a sparse solution is given in [11].

References

- [1] A. Ghosh, *The little known story of $F = ma$ and beyond*, Resonance 14, 1153 (2009).
- [2] S. H. Strogatz, *Nonlinear Dynamics And Chaos: With Applications To Physics, Biology, Chemistry, And Engineering*, Westview Press, Boulder (2014).
- [3] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press (2016).
- [4] T. M. Mitchell, *Machine Learning*, McGraw-Hill, New York (1997).
- [5] S. L. Brunton, J. L. Proctor and J. N. Kutz, *Discovering governing equations from data by sparse identification of nonlinear dynamical systems*, Proceedings of the National Academy of Sciences, 113(15), 3932-3937 (2016).
- [6] E. N. Lorenz, *Deterministic nonperiodic flow*, Journal of the Atmospheric Sciences, 20(2), 130141 (1963).
- [7] J. Pathak, B. Hunt, M. Girvan, Z. Lu, and E. Ott, *Model-Free Prediction of Large Spatiotemporally Chaotic Systems from Data: A Reservoir Computing Approach*, Phys. Rev. Lett. 120, 024102 (2018).
- [8] B. Van der Pol, *On relaxation-oscillations*, The London, Edinburgh and Dublin Phil. Mag. & J. of Sci., 2(7), 978992 (1926).

- [9] O. E. Rössler, *An Equation for Continuous Chaos*, Physics Letters, 57A(5), 397398(1976).
- [10] H. Goldstein, *Classical Mechanics*, Addison-Wesley Publishing Company (1980).
- [11] J. H. Friedman, R. Tibshirani and T. Hastie, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, New York : Springer, (2009).